

ACTIVIDAD 3



Desarrollo full stack

Tareas

Diseño de backend con reglas de negocio y control de estados

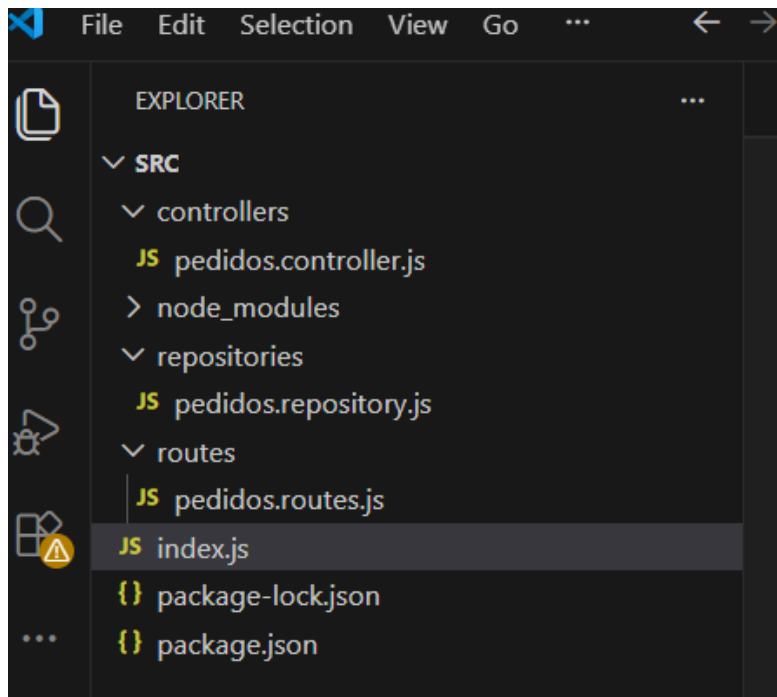
Isaac Isaias Betance

Introduccion

En esta actividad se desarrollo un backend con Express enfocado no solo en hacer un CRUD basico, sino en aplicar reglas de negocio reales como se hace en un proyecto real. El objetivo fue manejar pedidos con distintos estados y limitar ciertas acciones dependiendo de ese estado.

Objetivo del proyecto

- Implementar un backend usando Express
- Aplicar reglas de negocio claras
- Separar responsabilidades usando routes, controllers y repositories
- Usar Programacion Orientada a Objetos
- Manejar estados y validaciones sin base de datos



Entidad Pedido

Un pedido cuenta con los siguientes atributos:

- id
- producto
- cantidad
- estado

Estados permitidos:

- pendiente
- confirmado
- cancelado

Reglas de negocio implementadas

Creacion de pedidos

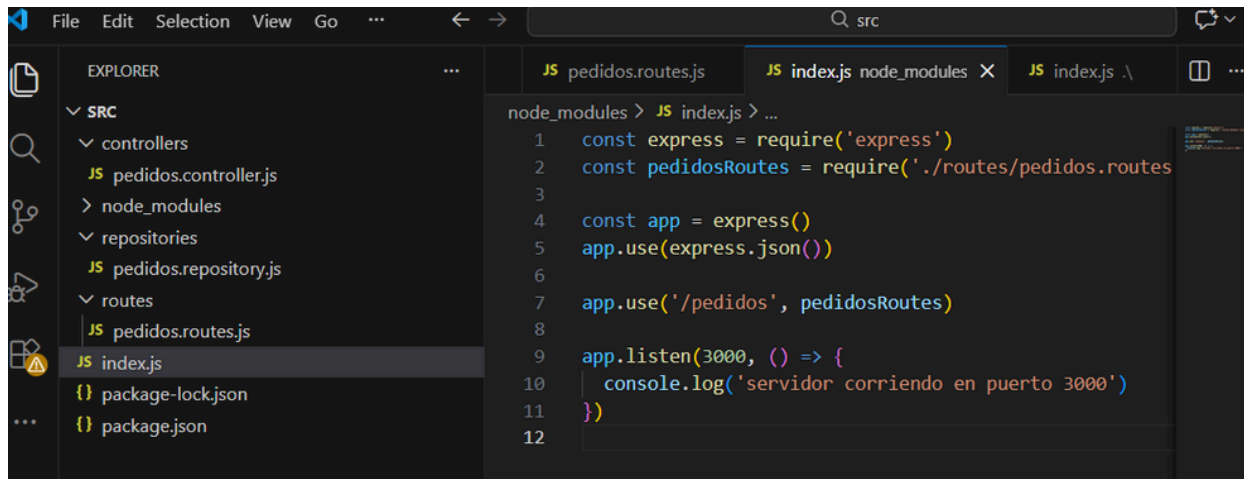
- Todo pedido nuevo inicia automaticamente en estado pendiente
- No se permite crear pedidos con cantidad menor o igual a 0
- Si la cantidad no es valida el sistema responde con error 400
- Request POST en Postman creando un pedido correctamente

Cambio de estado

- Un pedido pendiente puede cambiar a confirmado o cancelado
- Un pedido confirmado no puede modificarse
- Un pedido cancelado no puede modificarse
- Si se intenta modificar un pedido no permitido se regresa error 400 con un mensaje claro
- PUT valido cambiando pedido pendiente a confirmado
- PUT invalido intentando modificar pedido confirmado

Eliminacion de pedidos

- Solo se pueden eliminar pedidos en estado pendiente
- No se permite eliminar pedidos confirmados o cancelados
- El sistema responde con error 400 cuando la accion no es valida
- DELETE invalido sobre pedido confirmado



Endpoints implementados

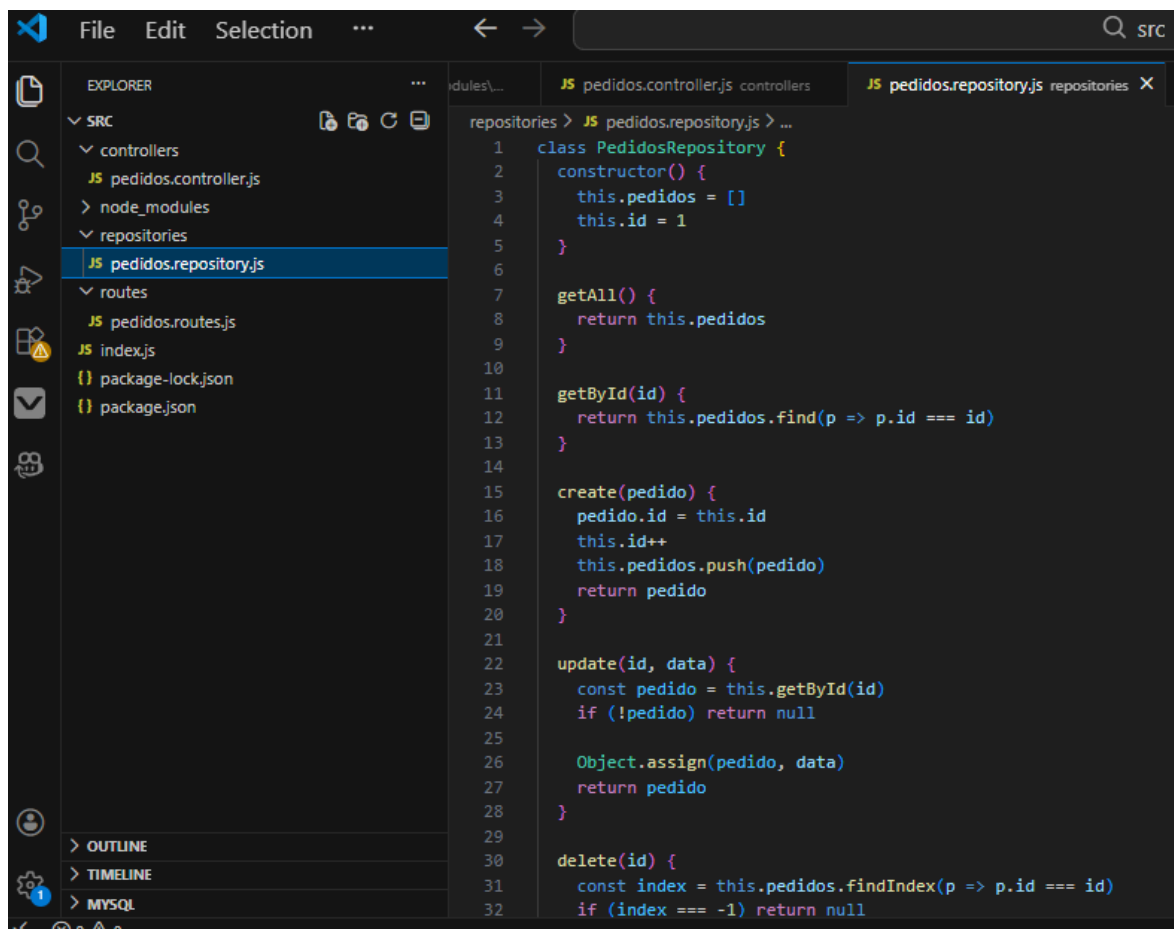
- GET /pedidos
- GET /pedidos/:id
- POST /pedidos
- PUT /pedidos/:id
- DELETE /pedidos/:id

Todos los endpoints responden con codigos HTTP correctos segun el escenario.

Estructura del proyecto

/src

- routes
- pedidos.routes.js
- controllers
- pedidos.controller.js
- repositories
- pedidos.repository.js index.js



Separacion de responsabilidades

Routes

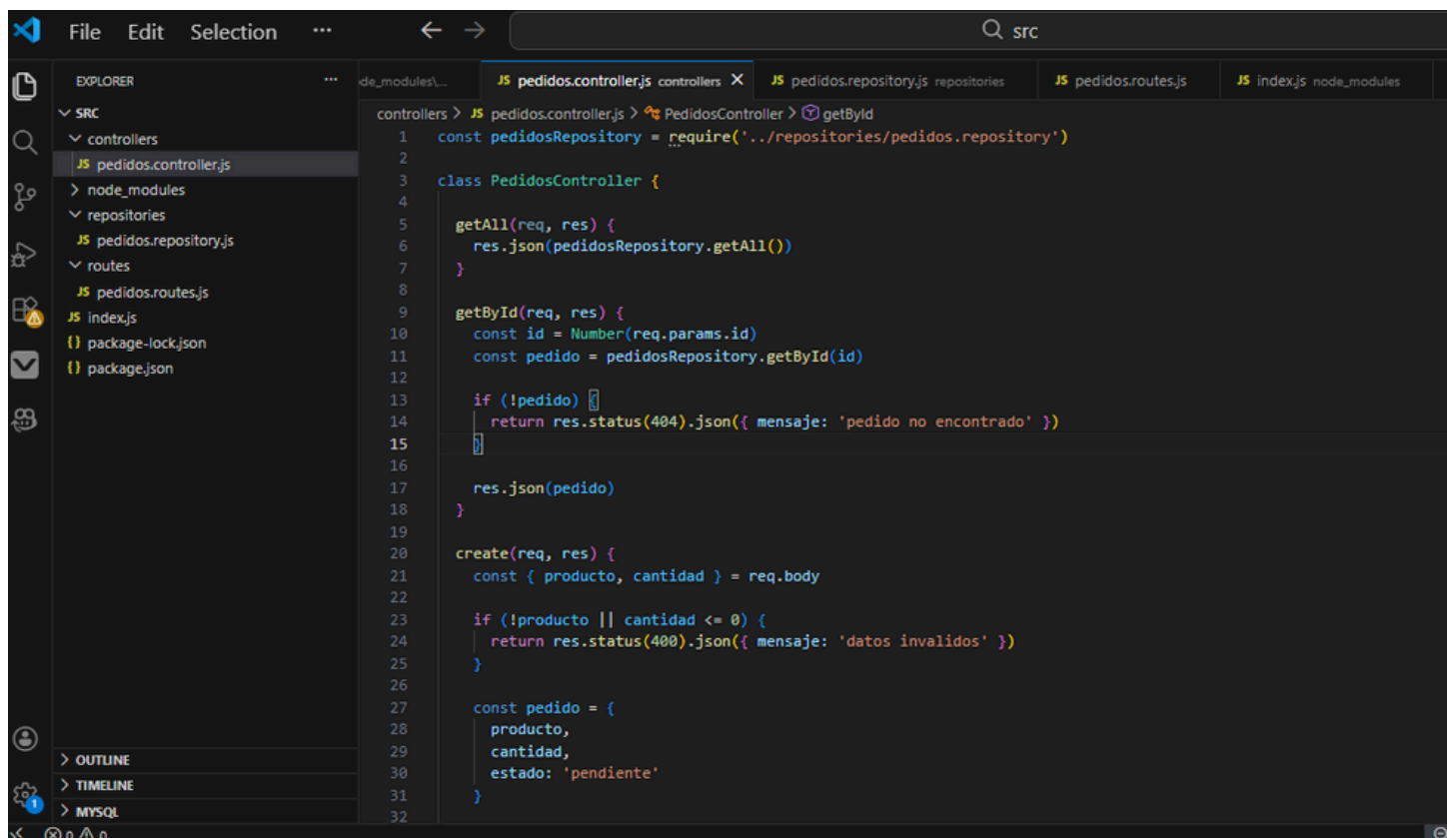
- Solo redirigen las peticiones
- No contienen logica

Controller

- Valida datos de entrada
- Aplica reglas de negocio
- Decide codigos HTTP y mensajes

Repository

- Maneja el almacenamiento en memoria
- Encapsula los datos usando una clase
- Usa constructor, metodos y this



Uso de Programacion Orientada a Objetos

- El repository fue implementado como una clase
- Se uso constructor para inicializar los pedidos
- Los metodos controlan el acceso y modificaciones
- Se utilizo this para manejar el estado interno

Repositorio en GitHub

Link al repositorio del proyecto :

GitHub - Yolsaac/backend-pedidos-reglas-negocio: Actividad 3 de desarrollo fullstack



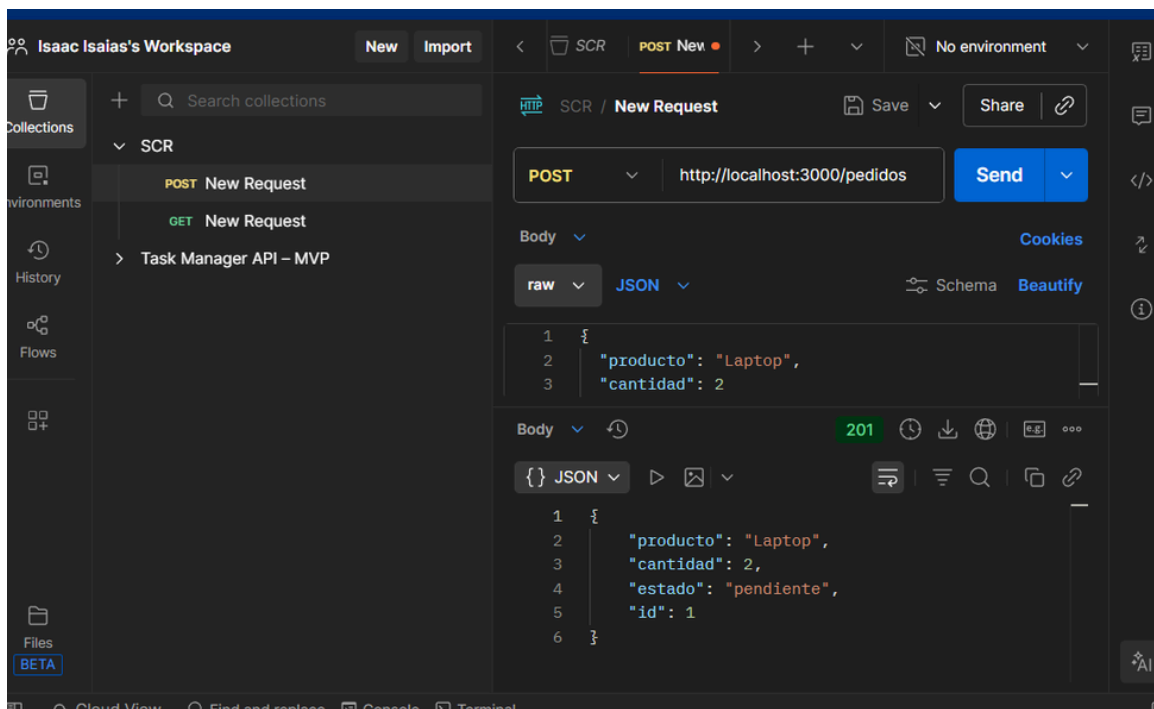
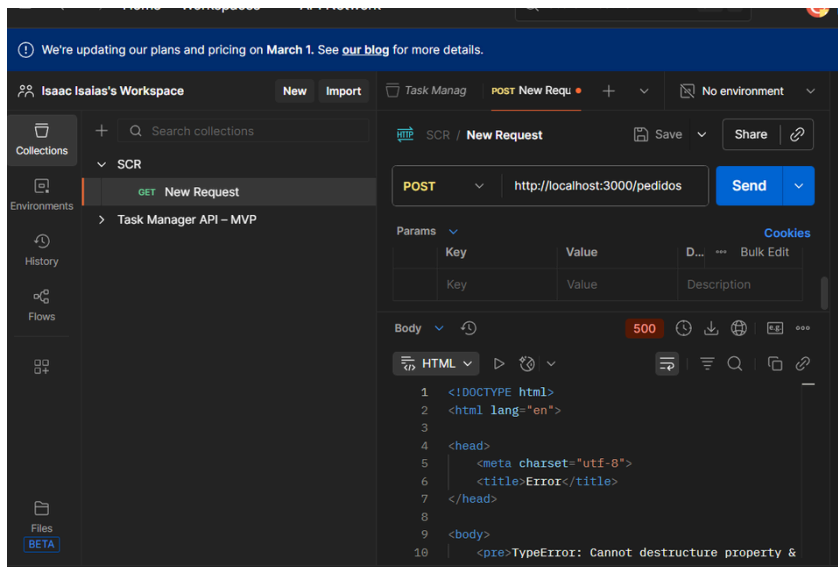
Evidencia en Postman

Se documento el funcionamiento del backend en Postman mostrando:

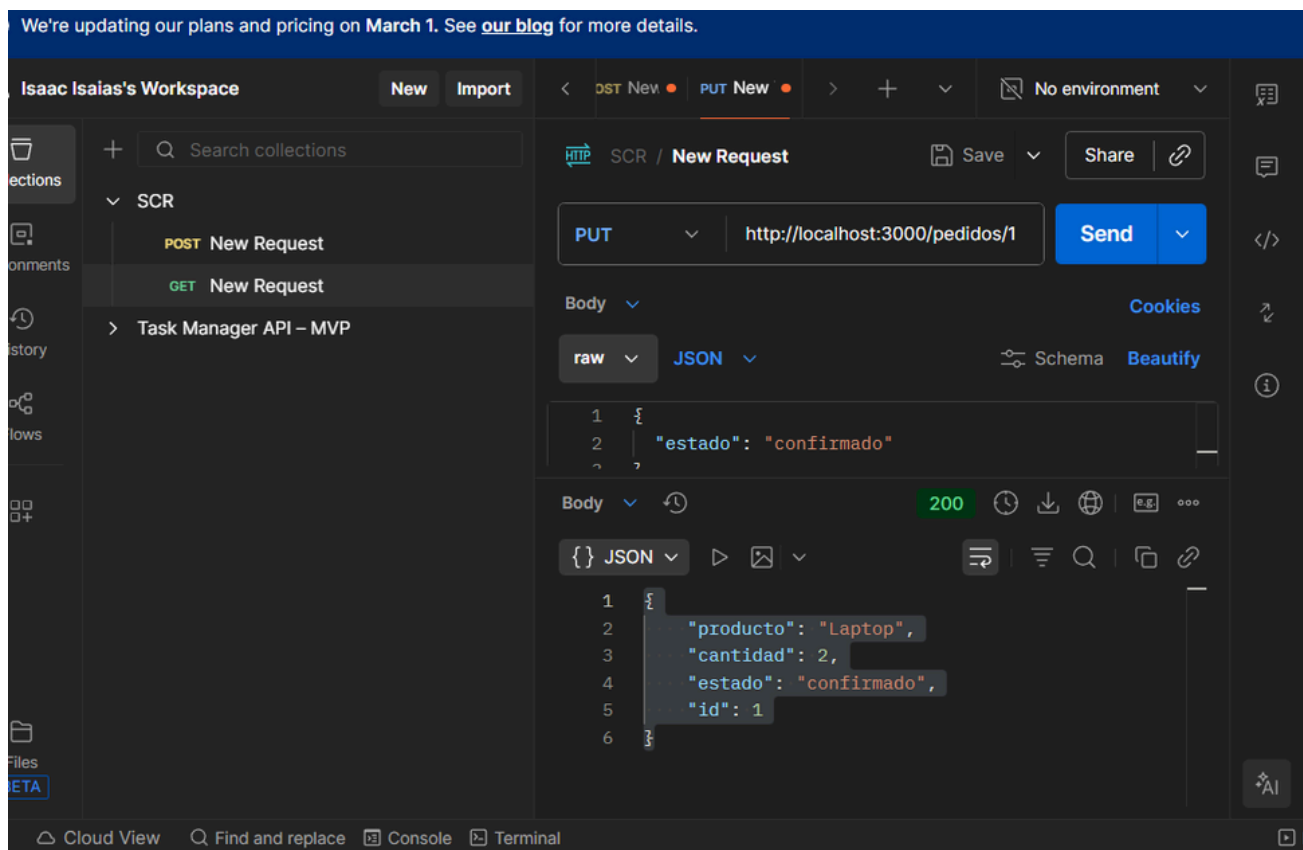
- Creacion correcta de pedido
- Intento de modificacion invalida
- Intento de eliminacion invalida

Link a la coleccion de Postman : <https://isaacisaia290-2346232.postman.co/workspace/Isaac-Isaias's-Workspace~6556038f-ac03-4438-8030-4b09e7187eb7/collection/51943433-308aae5d-c441-47f9-9213-aaa2c154a382?action=share&source=copy-link&creator=51943433>

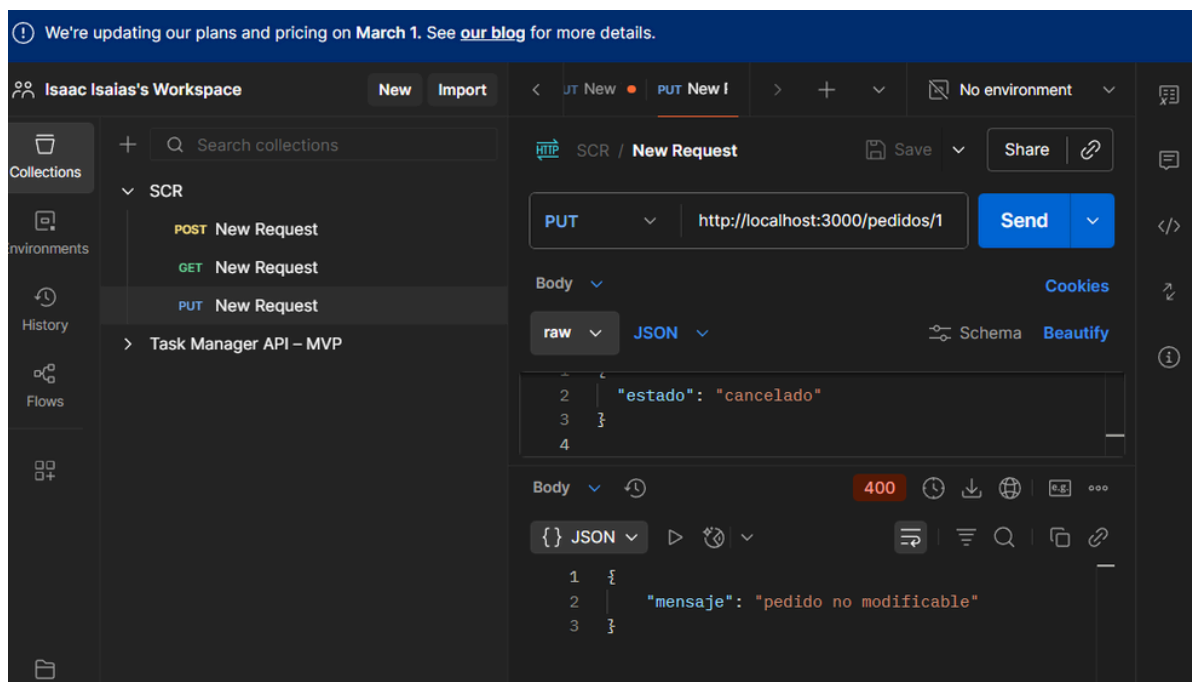
Creación correcta de pedido



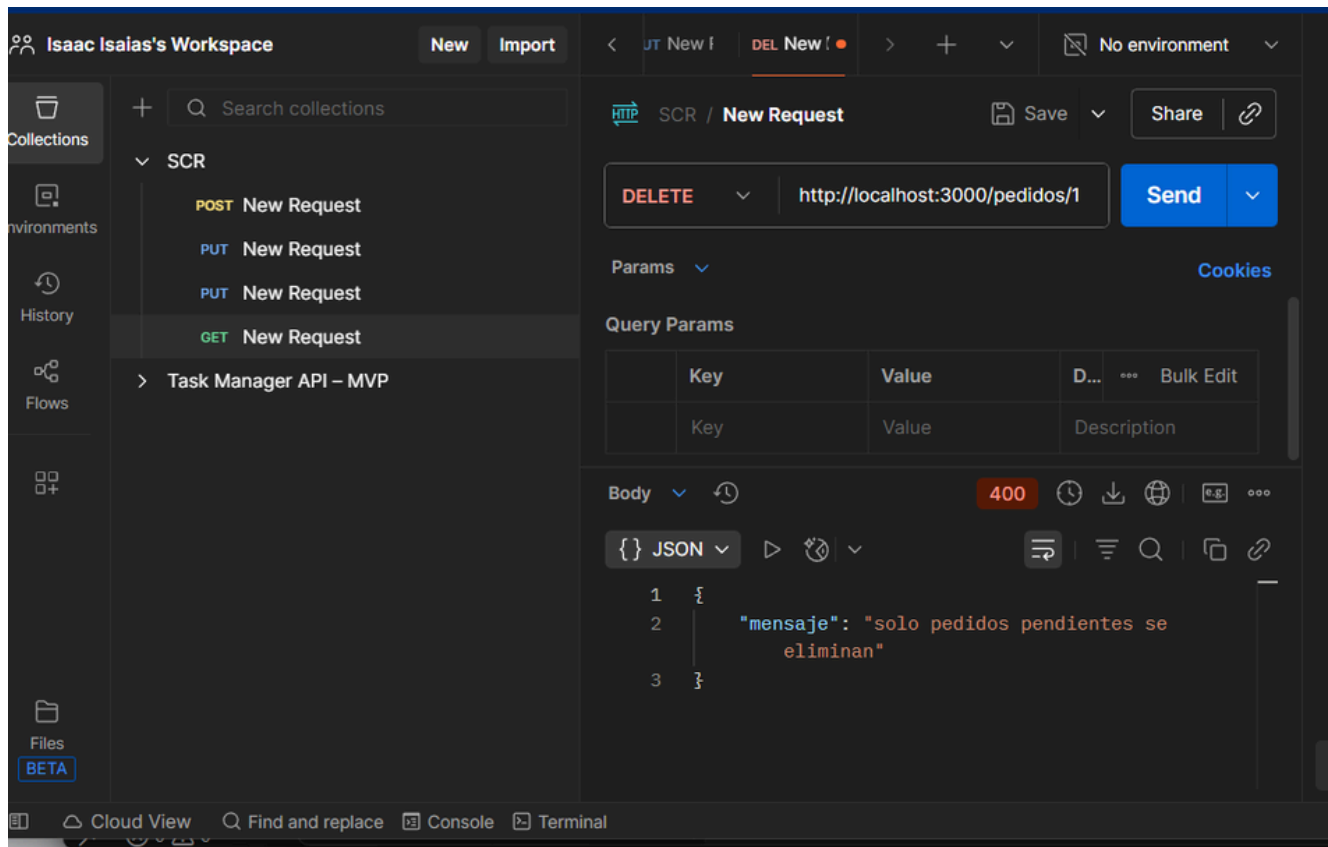
Cambio de estado válido



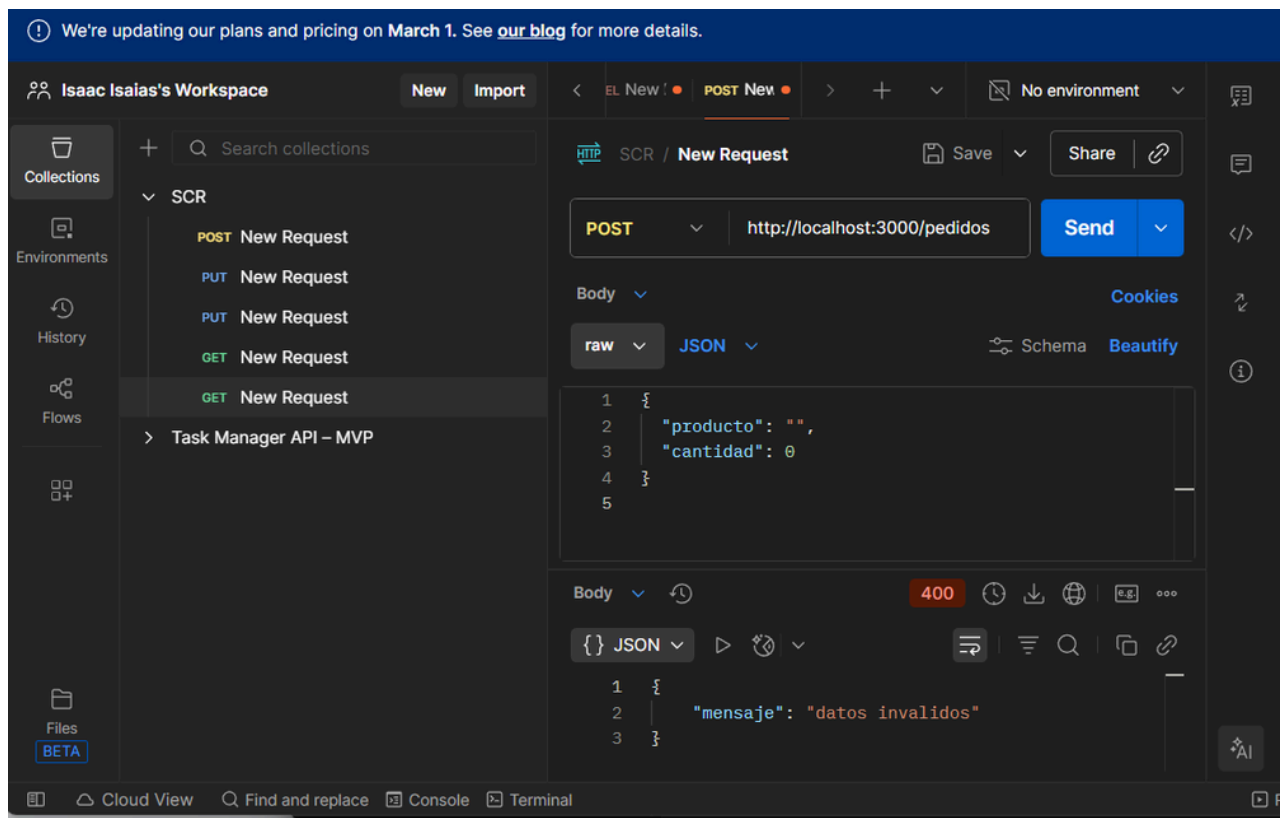
Modificación inválida



Eliminación inválida



Consulta general



<https://isaacisaias290-2346232.postman.co/workspace/Isaac-Isaias's-Workspace~6556038f-ac03-4438-8030-4b09e7187eb7/collection/51943433-308aae5d-c441-47f9-9213-aaa2c154a382?action=share&source=copy-link&creator=51943433>

Conclusion

Este proyecto permitio entender como funcionan las reglas de negocio dentro de un backend real. Separar responsabilidades y controlar estados ayuda a mantener un codigo mas claro, seguro y facil de escalar en el futuro.